

Task 18: Deploy Twenty CRM on Kubernetes (Local)

Objective

Deploy the Twenty CRM application on a local Kubernetes cluster (Docker Desktop Kubernetes), verify it is running, scale it, and clean up resources using the existing Twenty CRM Docker image. No registry push is required.

Environment

- Kubernetes: Docker Desktop Kubernetes (context: docker-desktop)
- Image used: twentycrm/twenty-app-dev:latest (existing local image, pulled via docker compose in earlier tasks)
- Namespace: devops-crm (created manually)

1. Enable Kubernetes

Kubernetes was enabled through Docker Desktop → Settings → Kubernetes → Enable Kubernetes.

The setup was verified with:

```
kubectl config current-context
kubectl get nodes
```

2. Create Namespace

The namespace was created manually rather than through a YAML manifest. This keeps the Kubernetes resources for the task isolated under devops-crm.

```
kubectl create ns devops-crm
```

Verified with:

```
kubectl get ns
```

3. Kubernetes Manifests

Two YAML manifests were used. The Deployment manages the Twenty CRM pod, while the Service exposes the application through a NodePort.

3.1 deployment.yaml

Explanation: This Deployment tells Kubernetes how to run Twenty CRM. It creates one pod using the existing local image, exposes container port 2020, and sets the SERVER_URL environment variable. imagePullPolicy: IfNotPresent allows Docker Desktop Kubernetes to use the image already available locally instead of requiring a registry pull.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: twenty-crm
  namespace: devops-crm
```

```

labels:
  app: twenty-crm
spec:
  replicas: 1
  selector:
    matchLabels:
      app: twenty-crm
  template:
    metadata:
      labels:
        app: twenty-crm
    spec:
      containers:
        - name: twenty-crm
          image: twentycrm/twenty-app-dev:latest
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 2020
          env:
            - name: SERVER_URL
              value: "http://localhost:2020"

```

3.2 service.yaml

Explanation: This Service provides network access to the Twenty CRM pod. It selects pods with the app: twenty-crm label and forwards traffic from Service port 2020 to container port 2020. The NodePort 30020 makes the application reachable from the local machine at http://localhost:30020.

```

apiVersion: v1
kind: Service
metadata:
  name: twenty-crm-service
  namespace: devops-crm
spec:
  type: NodePort
  selector:
    app: twenty-crm
  ports:
    - port: 2020
      targetPort: 2020
      nodePort: 30020

```

4. Deploy

The Deployment and Service were applied using:

```

kubectl apply -f deployment.yaml
kubectl apply -f service.yaml

```

5. Verify Pod Running

The pod and application startup were checked using:

```
kubectl get pods -n devops-crm
kubectl logs -n devops-crm <pod-name> -f
kubectl exec -n devops-crm <pod-name> -- netstat -tlnp
```

Startup was considered complete when:

- Pod status showed 1/1 Running with 0 restarts.
- Container logs showed s6 services (server, worker, cron) started successfully.
- Port 2020 appeared in the container's netstat output.

```

Image:          twentycrm/twenty-app-dev:latest
Image ID:       docker-pullable://twentycrm/twenty-app-dev@sha256:bb4a10e0b73bd89f65b1410e94753d32cbbd3f35d85aa1e51bc635dbc2fefe7
Port:          2020/TCP
Host Port:     0/TCP
State:         Running
Started:       Fri, 18 Sep 2026 17:21:50 +0530
Ready:         True
Restart Count: 0
Environment:
  SERVER_URL: http://localhost:2020
Mounts:
  /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-65kpd (ro)
Conditions:
  Type              Status
  PodReadyToStartContainers  True
  Initialized        True
  Ready              True
  ContainersReady    True
  PodScheduled       True
Volumes:
  kube-api-access-65kpd:
    Type:              Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName:      kube-root-ca.crt
    Optional:           false
    DownwardAPI:        true
QoS Class:           BestEffort
Node-Selectors:      <none>
Tolerations:         node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                    node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason      Age    From          Message
  ----    -
  Normal  Scheduled   75s    default-scheduler  Successfully assigned devops-crm/twenty-crm-b66dd787c-cvb6n to docker-desktop
  Normal  Pulled      68s    kubelet        spec.containers[twenty-crm]: Container image "twentycrm/twenty-app-dev:latest" already present on machine and c
an be accessed by the pod
  Normal  Created     67s    kubelet        spec.containers[twenty-crm]: Container created
  Normal  Started     66s    kubelet        spec.containers[twenty-crm]: Container started

C:\Users\Public\devopss\kubernetes>kubectl get po -n devops-crm
NAME              READY   STATUS    RESTARTS   AGE
twenty-crm-b66dd787c-cvb6n  1/1     Running   0          83s

C:\Users\Public\devopss\kubernetes>kubectl get svc -n devops-crm
NAME              TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)          AGE
twenty-crm-service  NodePort    10.111.128.31  <none>        2020:30020/TCP   20m

```

6. Scale to 2 Replicas

The Deployment was scaled from one replica to two replicas:

```
kubectl scale deployment twenty-crm -n devops-crm --replicas=2
kubectl get pods -n devops-crm
```

The result was two Twenty CRM pods in Running state. (See screenshot: scaled_2_replicas.jpeg)

```

C:\Users\Public\devopss\kubernetes>kubectl get po -n devops-crm
NAME              READY   STATUS    RESTARTS   AGE
twenty-crm-b66dd787c-cvb6n  1/1     Running   0          15m

C:\Users\Public\devopss\kubernetes>kubectl scale deployment twenty-crm -n devops-crm --replicas=2
deployment.apps/twenty-crm scaled

C:\Users\Public\devopss\kubernetes>kubectl get po -n devops-crm
NAME              READY   STATUS    RESTARTS   AGE
twenty-crm-b66dd787c-7gvvq  0/1     ContainerCreating   0          2s
twenty-crm-b66dd787c-cvb6n  1/1     Running             0          15m

C:\Users\Public\devopss\kubernetes>kubectl get po -n devops-crm
NAME              READY   STATUS    RESTARTS   AGE
twenty-crm-b66dd787c-7gvvq  0/1     ContainerCreating   0          10s
twenty-crm-b66dd787c-cvb6n  1/1     Running             0          16m

C:\Users\Public\devopss\kubernetes>kubectl get po -n devops-crm
NAME              READY   STATUS    RESTARTS   AGE
twenty-crm-b66dd787c-7gvvq  0/1     ContainerCreating   0          22s
twenty-crm-b66dd787c-cvb6n  1/1     Running             0          16m

C:\Users\Public\devopss\kubernetes>kubectl get po -n devops-crm
NAME              READY   STATUS    RESTARTS   AGE
twenty-crm-b66dd787c-7gvvq  1/1     Running             0          49s
twenty-crm-b66dd787c-cvb6n  1/1     Running             0          16m

C:\Users\Public\devopss\kubernetes>

```

7. Scale Back to 1 Replica

The Deployment was then scaled back to one replica:

```
kubectl scale deployment twenty-crm -n devops-crm --replicas=1
kubectl get pods -n devops-crm
```

The result was a single remaining pod. (See screenshot: scaled_back_to_1_replica.jpeg)

```
C:\Users\Public\devopss\kubernetes>kubectl scale deployment twenty-crm -n devops-crm --replicas=1
deployment.apps/twenty-crm scaled

C:\Users\Public\devopss\kubernetes>kubectl get po -n devops-crm
NAME                                READY   STATUS    RESTARTS   AGE
twenty-crm-b66dd787c-7gvvq         1/1     Terminating    0          2m23s
twenty-crm-b66dd787c-cvb6n         1/1     Running        0          18m

C:\Users\Public\devopss\kubernetes>kubectl get po -n devops-crm
NAME                                READY   STATUS    RESTARTS   AGE
twenty-crm-b66dd787c-cvb6n         1/1     Running        0          18m

C:\Users\Public\devopss\kubernetes>
```

8. Cleanup

The Kubernetes resources were removed using:

```
kubectl delete deploy twenty-crm -n devops-crm
kubectl delete svc twenty-crm-service -n devops-crm
```

Confirmed no resources remained in the devops-crm namespace. (See screenshot: deleted_all_pods_svcs.jpeg)

```
C:\Users\Public\devopss\kubernetes>kubectl get deploy -n devops-crm
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
twenty-crm 1/1      1            1           20m

C:\Users\Public\devopss\kubernetes>kubectl delete deploy twenty-crm -n devops-crm
deployment.apps "twenty-crm" deleted from devops-crm namespace

C:\Users\Public\devopss\kubernetes>kubectl get svc -n devops-crm
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)          AGE
twenty-crm-service  NodePort    10.111.128.31 <none>        2020:30020/TCP   39m

C:\Users\Public\devopss\kubernetes>kubectl delete svc twenty-crm-service -n devops-crm
service "twenty-crm-service" deleted from devops-crm namespace
```

Key Learnings

- The twenty-app-dev all-in-one image has a long first-boot time due to workspace schema migration. Probe timing, or removing probes entirely for local/dev use, needs to account for this, or the liveness probe can kill the container mid-startup.
- imagePullPolicy: IfNotPresent lets Docker Desktop Kubernetes reuse a locally built or pulled image without needing a registry.
- Without a readinessProbe, Kubernetes marks the container Ready as soon as the process starts, not when the app is actually serving traffic. Therefore, manual verification using logs and port-forward was used to confirm real readiness before relying on the Service.